WhatsApp Team Inbox — Pending & Future Work

Last updated: 12 Oct 2026. This is the single place for everything that is **not finished yet** around the WhatsApp Team Inbox, plus how to add the next thing without breaking what exists.

Related: `docs/whatsapp-team-inbox.md` (what is built and how to use it). When something here gets done, **move it out of this file**, mark its row in the tracker there, and rebuild the PDFs (section 6).

---

1. Before going live — do these first

These are not new features; the code for steps 1–3 is written but has not been proven on production.

#	Task	Why it matters	Done?
A1	Deploy, then run `php artisan migrate` (four migrations: `2026_10_13_100000`, `2026_10_14_100000`, `2026_10_15_100000`, `2026_10_16_100000`)	Without them the Inbox pages return 500 (missing tables / columns). **On Voroa this runs by itself:** `docker/start.sh` executes `php artisan migrate --force` on every start — after deploying, check the boot log for `migrate failed`	[ ]
A2	Switch on the "WhatsApp Team Inbox" module for a test tenant (plan editor, or Tenants → Modules)	The menu stays hidden and the URL gives 403 until then	[ ]
A3	Test step 1 on production: customer message arrives, appears in the Inbox, reply reaches the phone	Everything else depends on this	[ ]
A4	Test step 2: assign, reply-claims-chat, `/quick reply`, private note (must NOT reach the phone), auto-assign, `whatsapp.view_all` removal	See the test list in the chat history / `docs/whatsapp-team-inbox.md`	[ ]
A5	Test step 3: Team Performance numbers, a real bulk send in Campaigns, Bulk Send status/source filter	Never run against real data yet	[ ]
A6	Re-run `php artisan db:seed --class=PlanSeeder` **only if** you want the stock Pro/Business/Enterprise plans to include the Inbox automatically; otherwise tick it by hand in the plan editor	New module defaults to "off" on every existing plan	[ ]

#	Task	Why it matters	Done?
A7	Meta app checks (when you use your own Meta app, not the Milan gateway): Webhook URL `https://<domain>/webhook/whatsapp`, field **messages** subscribed, app in **Live** mode, Business Verification done	Without these, real customers' messages never reach the Inbox	[ ]
A8	Custom-role tenants: the owner must tick `whatsapp.inbox`, `whatsapp.send`, `whatsapp.assign`, `whatsapp.view_all` on their custom roles (only `tenant_admin` and `staff` get them automatically)	Otherwise those users see nothing / get 403	[ ]
A10	Test step 3b on production: (a) send STOP from a phone → confirmation arrives, number shows "Opted out" in Bulk Send and is skipped; START undoes it; (b) message from a brand-new number → a Lead "WhatsApp" appears and is linked in the chat; (c) schedule a bulk send 3 minutes ahead → it goes out and appears in Campaigns; cancel another one	Never run against real WhatsApp yet	[ ]
A11	**The scheduler is not started anywhere in this repo.** `docker/start.sh` starts only php-fpm and nginx — there is no cron / `schedule:run` / `schedule:work`. Check whether Voroa runs a separate scheduled job for `php artisan schedule:run` every minute. If not, scheduled WhatsApp sends — and also the appointment / subscription / invoice reminders and the lead SLA check — never run. Fix: add a once-a-minute `schedule:run` in the Voroa dashboard, or start `php artisan schedule:work &` in `docker/start.sh` before `php-fpm` (costs CPU on a small instance, so a platform cron is preferable)	Scheduled sends silently never go out	[ ]
A9	Find the unexplained **404** seen in the browser console on the Inbox page (an asset URL that was cut off in the screenshot)	Probably a layout icon/image, not the Inbox — but never confirmed	[ ]

---

2. Pending features (ordered by how much they help)

Each item: **what**, **why**, **effort** (S = under a day, M = 1–3 days, L = more), **where in the code**, **how**.

P1. "Read" status and delivery tracking on every connection — Effort M

- **Today:** status is only `sent` / `delivered` / `failed`. The gateway webhook (`WhatsappGatewayWebhookController`, status handler around the `delivered`/`read` check) treats *read* as *delivered*. The direct-Meta webhook (`WhatsappWebhookController::handle`) ignores status events completely, so on a direct-Meta connection every message stays `sent`.

- **Why:** campaign dashboard cannot show Delivered / Read for direct-Meta tenants; "read" is never shown at all.
- **How:** (1) widen the `whatsapp_logs.status` values to add `read` — copy the pattern of migration `2026_10_11_100000_add_received_status_to_whatsapp_logs.php` (MySQL enum vs PostgreSQL CHECK constraint; production is PostgreSQL, so do the constraint swap by hand). (2) In `WhatsappWebhookController::handle` also read `$value['statuses']` and update `whatsapp_logs` by `wamid` (same update logic as the gateway controller). (3) Stop folding read into delivered in the gateway controller. (4) Add a "Read" column to `campaigns.blade.php` / `campaign-show.blade.php` and the status chip list.

P2. Re-send to failed numbers from a campaign — Effort M

- **Today:** Campaign details shows why messages failed, but you cannot retry.
- **Why not done:** `whatsapp_logs` stores the *rendered text* only. For an approved Meta template the template name/language/params are not saved, so a faithful re-send is not possible; a plain-text re-send only works inside the 24-hour window.
- **How:** add `template_name`, `template_language`, `template_params` (json) columns to `whatsapp_logs`, fill them in `WhatsappController::sendBulk`, then add a "Retry failed" button on `campaign-show` that re-sends only `status='failed'` rows with the same `bulk_id` (create new logs under a new `bulk_id` or the same one with a `retry_of` marker).

P3. Show customers' incoming photos / documents / voice notes — Effort M

- **Today:** inbound non-text messages are logged as the label `[image]`, `[document]`, etc. (`WhatsappInboundService::handle`). The webhook parser (`WhatsappWebhookController::processMessage`) only reads text and button/list replies.
- **How:** read the media id from the webhook payload (`$message['image']['id']` etc.), fetch the file through the Graph API (or the gateway's media endpoint), store it (private disk), save `media_type` + a stored path on the log, and render it in the Inbox bubble (`bubbleHtml` in `inbox.blade.php`). Remember file size limits and that Meta media URLs expire after minutes.

P4. Pause the chatbot while a human is handling the chat — Effort S

- **Today:** the chatbot keeps answering keywords even after an agent has replied (`WhatsappInboundService::handle` always calls `WhatsappChatbotService`).
- **How:** in `handle()`, skip the chatbot when the conversation has `assigned_to` set AND the last outbound message with `sent_by` is within N minutes (make N a setting on `whatsapp_settings`). Add a tiny "Bot paused" hint in the chat header.

P5. Start a chat from a Lead / Contact — Effort S

- **Today:** a thread exists only after the customer writes first (`WhatsappConversation::touchFor` returns null for outbound-only numbers). Staff cannot open a chat with a lead from the Inbox.
- **How:** add a "Message in Inbox" button on the Lead / Contact page that creates the conversation (`touchFor(... $inbound = false)` with a `$force` flag) and opens `inbox?c=<id>`. Outside the 24-hour window the template picker already handles the first message.

P6. New-message alerts (sound, browser title, push) — Effort S–M

- **Today:** the page refreshes every 6 s but there is no sound or notification; if the tab is in the background nobody notices.
- **How:** in `inbox.blade.php` `poll()`, compare unread counts between polls → play a short sound and set `document.title = "(2) WhatsApp Inbox"`. For real push, reuse the existing notification system (`NotificationService`, `DeviceToken`) and fire a notification to the assignee in `WhatsappConversation::touchFor`.

P7. Edit / delete private notes — Effort S

- **Today:** notes cannot be changed or removed. Add `PUT` / `DELETE inbox/notes/{id}` (author or admin only) and small / x controls on the note card.

P8. Smarter auto-assign — Effort M

- **Today:** plain round-robin among ticked users; does not know who is on leave or online (`WhatsappConversation::nextAutoAssignee`).
- **How:** add an "Available" toggle per user (or use existing attendance data) and skip unavailable people; optionally cap open chats per person.

P9. Reports — Effort S each

- Export the performance report / a campaign's recipients to CSV (`WhatsappInsightsController`; there is an existing `app/Exports` folder for the pattern).
- "Resolved by" and time-to-resolve: add `resolved_by` / `resolved_at` to `whatsapp_conversations`, set them in `WhatsappInboxController::toggleStatus`, then show chats resolved per agent.
- Business-hours-aware first-reply time (ignore night hours) using the existing `whatsapp_settings` business hours.

P10. Labels / tags on chats and on Leads & Contacts — Effort M

- **Today:** Leads and Contacts have **no tags column** (only Tasks do), so segmentation by tag is impossible. Bulk Send can filter Leads by status and source only; Contacts by search text only.

- **How:** add a `tags` json column to `leads` and `contacts` (see `2026_08_12_120328_add_tags_to_tasks_table.php` for the pattern), a tag editor on the Lead/Contact form, then a tag filter in `bulk.blade.php` (`applyFilters()` already supports data attributes). Chat labels would need their own table.

P11. Real-time instead of polling — Effort L

- Polling every 6 s is fine up to a few dozen users per tenant. If load grows, move to Laravel Reverb / Pusher (broadcast an event from `WhatsappConversation::touchFor`) and subscribe in `inbox.blade.php`. Do this only when polling becomes a measurable cost.

P12. Product Catalog Sharing (original step 4) — Effort L, blocked

- **What:** send a WhatsApp catalog / product cards from the Inbox and Bulk Send.
- **Blocked by:** (1) your own **verified Meta app** (today the CRM uses the Milan gateway); (2) Meta **App Review** for `catalog_management` and `business_management`; (3) a catalog in Meta Commerce Manager connected to the customer's WhatsApp Business Account. Also check whether the Milan gateway supports catalog messages at all (not checked).
- **How:** store `catalog_id` on `whatsapp_settings`; add `sendCatalog()` / `sendProduct()` (interactive `catalog_message` / `product`) to `WhatsappChatbotService` next to `sendInteractiveButtons`; new module key `whatsapp_catalog` in `config/modules.php`; buttons in the Inbox composer.
- **Build it when:** a paying customer asks for it. Until then product image / PDF / link already works through chatbot flows and Inbox attachments.

P13. Opt-out polish — Effort S

- Opt-out list page (view / search / remove numbers) and CSV import of numbers who already refused marketing elsewhere.
- Custom STOP/START words and custom confirmation text per tenant (words are fixed in `WhatsappOptOut::STOP_WORDS` / `START_WORDS`, e.g. add Hindi/Gujarati words like "band").
- Opt-in capture (record *when* and *how* consent was given) if a customer needs proof for audits.

P14. Scheduling polish — Effort S–M

- Edit a scheduled send (today: cancel and create again), recurring sends (weekly offer), time-zone per tenant (today: server timezone, India), and an email / notification when a scheduled send finishes or fails.

P15. Auto-lead polish — Effort S

- Choose the lead's default status / owner / source text per tenant, skip auto-lead for numbers that only message a certain keyword, and a "WhatsApp leads" quick filter on the Leads page.

3. Known limits (by design for now)

- A chat appears only after the customer writes (see P5).
- Opt-out blocks **bulk** sends only. Automatic reminders (appointments, subscriptions, invoices, loyalty) and one-to-one replies still go out — decide per customer whether reminders should respect opt-out too (not done).
- Scheduled recipients are fixed when you schedule; leads added later are not included. A recipient deleted before the time is silently skipped.
- Replying to an unowned chat makes it yours; there is no "release back to pool" besides choosing *Unassigned* in the dropdown.
- Delivered counts exist only for gateway-linked numbers (see P1).
- First-reply time counts only replies typed by a person; chatbot / automatic / bulk messages are ignored, and a customer's later messages in an already-waiting chat do not restart the clock.
- Reports scan at most 100,000 messages per request (`WhatsappInsightsController::MAX_ROWS`); the page warns when it hits the cap.
- The Inbox history is built from `whatsapp_logs`; deleting logs removes chat history.

4. Not planned (from the SandeshChat comparison)

- Shopify / WooCommerce integration and custom workflow triggers on WhatsApp events — not built; would be a separate project (the CRM already has outgoing webhooks and an API that can be used meanwhile).
- Multiple WhatsApp numbers per tenant in one Inbox — the data model assumes one connected number per tenant.

5. Adding the next feature — checklist

1. **Decide the gate.** Premium for some customers → a key in `config/modules.php` (it appears automatically in the plan editor, the per-tenant toggle and PlanSeeder). Role-level control → a permission.
2. **Permission:** add it to `database/seeders/RolesAndPermissionsSeeder.php` (fresh installs) **and** to a new migration that creates it and grants it to `tenant_admin` / `staff` (existing installs) — copy `2026_10_15_100000_add_view_reports_permission_to_whatsapp_inbox.php`.

3. **Routes:** `routes/web.php`, inside the `whatsapp` prefix group, behind `module:whatsapp_inbox` and `permission:…`.
4. **Data access:** inside the Inbox controller always go through `scoped()` / `conversation()` so the `whatsapp.view_all` rule is respected; in webhooks (no logged-in user) always filter by `tenant_id` explicitly.
5. **UI:** the Inbox page is JSON-driven (`inbox.blade.php`); add an endpoint that returns JSON and update the page without reloading. Avoid `{{ }}` inside JS template strings (Blade swallows them) and do not reuse global names from the layout (`CSRF` is already taken by the notification bell — wrap page scripts in an IIFE).
6. **Test cheaply before deploy:** `php -l` the files, `php artisan route:list --name=whatsapp`, `php artisan view:cache`, `php artisan migrate --pretend`.
7. **Docs:** update `docs/whatsapp-team-inbox.md` (tracker + how-to-use + seller notes + developer notes), the three guides in `docs/full-documentation/` if user-visible, move the item out of this file, then rebuild the PDFs (below).

6. Rebuilding the PDFs

Run from the project root (uses packages already installed; no Docker needed):

```
php docs/tools/make-pdf.php . docs "whatsapp-team-inbox|docs/whatsapp-team-inbox.md"
"whatsapp-pending-and-future-work|docs/whatsapp-pending-and-future-work.md"
php docs/tools/make-pdf.php . docs/full-documentation "seller-guide|docs/full-
documentation/seller-guide.md" "user-guide|docs/full-documentation/user-guide.md"
"developer-guide|docs/full-documentation/developer-guide.md"
```

Emoji are removed automatically; tables and headings are kept. Open one PDF after building to check the layout.

7. Change log

Date	Change
11 Oct 2026	Step 1 built: shared inbox, reply, 24-hour window, resolve, Lead/Contact link, plan/permission gating
12 Oct 2026	Step 3b built: broadcast opt-out (STOP/START + manual), automatic Lead from first WhatsApp message, scheduled bulk sends (with `whatsapp:send-scheduled` command); bulk sending moved into `WhatsappBulkSender`
12 Oct 2026	Inbox made JSON-driven (no page reloads); step 2 built (assign, auto-assign, quick replies, notes); step 3 built (team performance, campaigns, Bulk Send filters); guides + PDFs updated; this file created